DYNAMIC PROGRAMMING

Ideas, Tactics, Patterns, and Problem-Solving Strategy

A concept-first guide for learning how to think about DP—not a worksheet of problems.

What this guide teaches

- How to recognize when dynamic programming may help
- How to define a state and write a transition
- How to choose memoization or tabulation
- How to reason about base cases, order of computation, and complexity
- How to identify common patterns: 1D, grid, knapsack, subsequence, interval, and tree DP
- How to optimize memory and debug incorrect recurrences

Designed for DSA study and interview preparation

Read the guide actively: for each recurrence, explain what the state means in one sentence before looking at the formula.

1. The Core Idea

Dynamic programming (DP) is a way to solve a problem by breaking it into smaller, related subproblems and reusing their answers. Instead of repeatedly solving the same subproblem, DP records its result and retrieves it when needed.

The two signals

• **Overlapping subproblems:** the same smaller question appears along multiple paths of the computation.

• **Optimal substructure:** an optimal answer can be constructed from answers to smaller subproblems. This is especially important for optimization tasks such as minimum cost, maximum value, or shortest paths.

A useful mental model

Imagine a large question as a network of smaller questions. Each node is a subproblem; an edge means one answer depends on another. DP computes each needed node once (or a bounded number of times) and reuses it.

DP is not automatically the right tool just because a problem asks for a maximum or minimum. Greedy algorithms, graph algorithms, binary search, or direct mathematics may be simpler. Look for repeated subproblems and a precise dependency structure.

Example idea: to count ways to reach step n when each move is 1 or 2 steps, the last move came from n–1 or n–2. Therefore, the number of ways to reach n depends on the answers for those two smaller states.

```
ways[n] = ways[n - 1] + ways[n - 2]
```

The formula is short; the important reasoning is why the final move divides all valid paths into two disjoint groups.

2. Recognizing DP in a Problem

Certain phrases often suggest DP, but they are clues rather than guarantees.

Common clues

• “How many ways…?” — counting constructions, paths, partitions, or sequences.

• “What is the minimum/maximum…?” — optimize a score, cost, number of operations, or value.

• “Can we reach/construct…?” — feasibility with choices and constraints.

• “Choose or skip”, “take or not take”, “end at index i” — a sequence of decisions.

• A recursive solution repeats calls with the same parameters.

• The answer for a prefix, position, capacity, or interval can be reused.

A recognition checklist

■ Can I describe a smaller version of the same problem?

■ Do different choices lead to the same smaller state?

■ Can I define a state using a small number of parameters?

■ Can the answer be expressed from smaller states?

■ Is the number of distinct states manageable?

When DP may be a poor fit

If each recursive branch reaches completely different subproblems, caching may not help. If a local choice can be proven to be globally safe, a greedy strategy may be simpler. If the state would require tracking nearly the entire history, rethink the representation or search for a stronger invariant.

A practical rule: first write a brute-force recursive definition. Then ask whether repeated states exist and whether the state can be compressed.

3. State Design: The Most Important Step

A DP state is a compact description of a subproblem. State design is often the hardest part: a correct state makes the transition almost obvious; an incomplete state loses information and causes incorrect answers.

Write the state in words first

Examples of useful state meanings:

• **dp[i]:** the best answer using the first i items.

• **dp[i]:** the best answer for a sequence that must end at index i.

• **dp[i][j]:** the answer using the first i items with capacity j.

• **dp[r][c]:** the number of ways or minimum cost to reach grid cell (r, c).

• **dp[l][r]:** the best answer for the interval from l through r.

• **dp[u][p]:** the best answer for the subtree rooted at u under condition p.

A state must preserve relevant information

Suppose future decisions depend on both the current index and how much capacity remains. An index-only state is insufficient because two paths can reach the same index with different remaining capacities and therefore have different futures.

State design questions

• What part of the input has already been processed?

• What information can affect future choices?

• Does the state mean “best answer among all prefixes” or “best answer ending exactly here”? Those are different meanings.

• Can two histories with the same state always share the same future calculation?

Avoid adding dimensions “just in case.” Every dimension should represent information that changes the answer. A small, meaningful state is usually faster and easier to prove correct.

4. Transitions and Recurrence Relations

A transition explains how a state is calculated from smaller states. It usually follows from considering the final action, the final element, or the last split in a solution.

The choice method

Ask: what could the last decision have been? List the possibilities, express each possibility using smaller states, and combine them according to the task.

• **Minimization:** take the minimum over valid choices.

• **Maximization:** take the maximum over valid choices.

• **Counting:** add counts for disjoint cases; avoid counting the same object twice.

• **Feasibility:** combine truth values with OR when any choice can work, or AND when all conditions must hold.

Generic transition shapes

```
dp[state] = min(cost(choice) + dp[next_state])  # minimum cost

dp[state] = max(value(choice) + dp[next_state]) # maximum value

dp[state] = sum(dp[smaller_state] for each valid case) # counting
```

The exact direction of the transition depends on whether the state is defined forward or backward. Both are valid if the meaning and base cases are consistent.

Do not guess the recurrence

Derive it from a complete case split. For example, if every valid solution must end with exactly one of several possible final actions, those cases should cover all valid solutions without overlap. This is the foundation of a correctness proof.

> **Check that every referenced state is smaller or already computable in the chosen order. If a transition depends on a state that has not been computed yet, either change the computation order or reconsider the recurrence.**

5. Base Cases, Boundaries, and Computation Order

Base cases are the smallest states whose answers can be determined directly. Every chain of dependencies must eventually reach a base case; otherwise the recurrence may loop or rely on undefined values.

Base-case habits

• Define what an empty prefix means.

• Check the first index and the smallest valid input.

• Distinguish an impossible state from a state whose answer is zero.

• For minimization, use an infinity-like sentinel carefully; never add to it without guarding against overflow.

• For counting, decide whether the empty construction counts as one way.

Topological order

A DP dependency graph is often a directed acyclic graph (DAG). Compute states in an order where all dependencies of a state are ready first. For prefix DP, this is often left to right; for interval DP, it is usually increasing interval length.

A simple dependency example

```
dp[0] = base_value
for i in range(1, n + 1):
    dp[i] = combine(dp[smaller states])
```

This is a template, not a universal loop. Some recurrences run backward, some use nested loops, and tree DP often uses a postorder traversal.

Boundary testing: try the smallest input, a single element, all equal values, strictly increasing or decreasing values, and a case where every choice is impossible. Many DP bugs are boundary bugs rather than deep algorithmic errors.

6. Memoization vs. Tabulation

Memoization: top-down

Start from a recursive definition. Before computing a state, check whether its answer is already stored. If so, return it; otherwise compute, store, and return the result.

```
solve(state):
    if state is a base case: return base answer
    if state is memoized: return memo[state]
    answer = combine(solve(smaller states))
    memo[state] = answer
    return answer
```

Advantages: often mirrors the recurrence, can skip unreachable states, and is natural when only some states are needed. **Risks:** recursion overhead, stack-depth limits, and accidental repeated work if caching is incomplete.

Tabulation: bottom-up

Create a table and fill states in dependency order, starting from base cases. Every transition reads already-computed entries.

```
initialize dp with base cases
for state in dependency order:
    dp[state] = combine(previously computed states)
```

Advantages: avoids recursion depth, makes iteration and memory optimization explicit, and can have lower overhead. **Risks:** it may compute states that are never needed, and the correct order may be less obvious.

Which should you choose?

Use memoization when the recursive structure is clearer or the reachable state space is sparse. Use tabulation when the order is simple, recursion depth is risky, or you want to reduce memory. Interview solutions can begin top-down for clarity and then be converted to bottom-up.

> **Both methods implement the same recurrence. They are not different mathematical algorithms when they compute the same states with the same transitions.**

7. Complexity: Count States and Transitions

A common DP complexity mistake is to report only the number of table entries. The time also depends on how much work is done to compute each entry.

The standard estimate

Time ≈ number of states × work per state. If each state checks several transitions, count those transitions too. **Space** includes the DP table, memoization cache, and any relevant recursion stack.

Illustrative examples

Pattern	Typical states	Typical work	Typical time
1D DP	n	constant per state	O(n)
Grid DP	rows × cols	constant per cell	O(rows × cols)
Knapsack	n × capacity	constant per state	O(n × capacity)
Interval DP	n^2 intervals	up to n split points	$O(n^3)$
Subset mask DP	2■ masks	often n choices	O(n × 2■)

A quick audit

• How many parameter combinations are possible?

• Can some combinations be skipped as unreachable?

• Does each transition scan all earlier positions or split points?

• Can the table be compressed without losing needed values?

Pseudo-polynomial complexity, such as O(nW) for capacity W, is polynomial in the numeric value W—not necessarily in the number of bits used to encode W. This distinction matters when W is large.

8. Common Pattern: 1D and Prefix DP

In 1D DP, a state is indexed by one position, count, or resource level. The main challenge is deciding whether the state describes the best answer for a prefix or an answer that must end at a particular position.

Prefix state vs. ending-here state

• **Prefix state:** dp[i] represents the best answer using the first i elements. The final element may be unused in the chosen solution.

• **Ending-here state:** dp[i] represents the best answer for a solution that must end at element i. This is useful when the last element is part of the answer.

These states can have different recurrences even when they refer to the same input. Be explicit about whether index i is included and whether the answer must end there.

Typical decisions

• Take or skip the current element.

• Extend a previous valid solution or start a new one.

• Choose among a small set of previous positions.

• Split a prefix into a final segment and an earlier prefix.

How to recognize it

Look for sequence tasks where the answer at a position depends on earlier positions, a bounded set of prior states, or the best answer for a prefix. If every state checks all previous positions, time may become $O(n^2)$; if only a constant number are needed, it may be O(n).

Do not force a 1D state if future decisions need extra information such as remaining capacity, the last chosen category, or the number of times an action has been used.

9. Common Pattern: Grid and Path DP

Grid DP commonly counts paths, finds minimum cost, or checks reachability. A cell is a natural state because the ways to arrive at that cell depend on a limited set of predecessor cells.

State and transition

A state such as dp[r][c] might mean “the number of ways to reach cell (r, c)” or “the minimum cost to reach cell (r, c).” These meanings are not interchangeable.

```
count[r][c] = count[r - 1][c] + count[r][c - 1]
```

This illustrative transition assumes movement only from above or from the left and that both predecessor cells are valid. Obstacles and boundaries require special handling.

Grid DP checklist

• Which moves are allowed?

• Does the state count paths, minimize cost, maximize reward, or record reachability?

• Does entering a cell incur a cost?

• How are obstacles represented?

• Can a row be computed using only the previous row?

Common mistakes

Adding a cell's cost at the wrong point can double-count or omit it. Treating an obstacle as an ordinary cell can create invalid paths. Incorrect initialization of the first row or column can contaminate every later state.

> **If moves can go both forward and backward in a way that creates cycles, ordinary topological grid DP may not apply directly. The movement graph and its dependency structure determine the right technique.**

10. Common Pattern: Knapsack and Resource DP

Knapsack-style DP appears when there are choices with limited resources: capacity, time, budget, weight, or number of operations. A common state tracks how many items have been considered and the resource limit or amount used.

A typical state

dp[i][w] can mean the maximum value obtainable from the first i items with capacity w. The transition compares skipping item i with taking it when its weight fits.

```
skip = dp[i - 1][w]
take = value[i] + dp[i - 1][w - weight[i]]
dp[i][w] = max(skip, take)  # if it fits
```

This form assumes each item can be used at most once and the previous row is used for both choices. Unbounded variants have different dependency rules.

Loop order matters

When compressing to one dimension, a 0/1 knapsack update typically iterates capacity downward so that an item is not reused during the same iteration. An unbounded knapsack update often iterates upward so the current item can contribute repeatedly. The correct direction follows from the intended recurrence.

Questions to ask

• Can an item be chosen zero, one, or many times?

• Is capacity an upper bound or must the exact amount be reached?

• Are negative values or zero weights possible?

• Is the goal maximum value, minimum number, number of combinations, or number of ordered sequences?

These variants can look almost identical while requiring different transitions or loop orders. Define the state and reuse rules before writing code.

11. Common Pattern: Subsequences and String DP

Subsequence DP often compares positions in one or two sequences. The state may track prefixes of two strings, the length of a best subsequence, or a position plus a constraint on what can follow.

Two-sequence state

For two strings, dp[i][j] can represent an answer using the first i characters of one string and the first j characters of the other. A transition often depends on whether the last characters match.

• If the characters match, a solution may extend the answer for the two shorter prefixes.

• If they differ, one or both final characters may be excluded and the resulting states compared.

• Base cases often involve an empty prefix.

Subsequence vs. substring

Subsequence: elements remain in order but need not be adjacent. **Substring/subarray:** elements must be contiguous. Confusing these constraints changes the state and recurrence.

Other common designs

• Longest increasing subsequence: state can describe the best sequence ending at index i.

• Edit distance: state can describe the minimum edits to transform one prefix into another.

• Sequence matching: state can track positions in both sequences.

For two-dimensional prefix DP, carefully determine whether i and j are lengths or zero-based indices. Mixing these conventions is a frequent source of off-by-one errors.

12. Advanced Families: Interval, Tree, and Bitmask DP

Interval DP

The state describes a contiguous interval, often dp[l][r]. The transition tries a final split point k or a final operation. Because shorter intervals are dependencies of longer intervals, compute by increasing interval length.

```
for length in increasing order:
    for each interval [l, r] of that length:
        try each valid split k
        dp[l][r] = best candidate
```

There are $O(n^2)$ intervals and up to $O(n)$ split points per interval, leading to $O(n^3)$ time in a common form.

Tree DP

The state describes a node's subtree and sometimes a condition such as whether the node is selected. A node's answer is combined from its children, usually using DFS in postorder so child answers are ready first.

Bitmask DP

A bitmask encodes a subset of items. dp[mask] or dp[mask][last] can represent an answer for the selected subset, optionally ending at a particular item. It is powerful for small n because 2■ subsets exist, but becomes expensive as n grows.

General lesson

These patterns differ in what a state represents and in the natural computation order. Interval DP is organized by length, tree DP by subtree dependencies, and bitmask DP by subset transitions. Identify the structure before selecting loops.

> **If the number of states is exponential, verify that the input constraints really are small enough. “DP” does not automatically mean efficient for large inputs.**

13. Space Optimization and Reconstruction

Rolling arrays

If dp[i] depends only on dp[i–1] or a fixed number of earlier rows, storing the entire table may be unnecessary. Keep only the rows or values still needed by future transitions.

• For a grid, retain the previous row and the current row if dependencies allow it.

• For a simple 1D recurrence, a few variables may replace an O(n) array.

• For knapsack, a one-dimensional array can reduce O(nW) space to O(W), but update order must preserve the original dependency semantics.

Can the answer still be reconstructed?

A compressed table may preserve the optimal value but lose the decisions that produced it. If the task asks for the actual chosen items or path, store parent decisions, retain enough table information, or design a reconstruction pass.

Memory trade-offs

Memoization can store only reached states, but hash maps and recursion add overhead. Tabulation often uses compact arrays but may fill every state. The best choice depends on state density, dimensions, constraints, and whether reconstruction is required.

> **Before optimizing space, first make the full-table recurrence correct. Then identify exactly which prior states are read and overwrite values only when they are no longer needed.**

14. Proving Correctness and Debugging DP

A short correctness proof

• **State meaning:** define precisely what each state represents.

• **Base cases:** show the smallest states are correct.

• **Transition:** show every valid solution is covered by the choices and no invalid solution is introduced.

• **Order:** show every dependency is computed before it is used.

• **Conclusion:** identify the state that contains the requested answer.

Debugging checklist

■ Does the state meaning match the code comments?

■ Are all base cases correct for empty and minimal inputs?

■ Are impossible states distinct from valid zero-valued states?

■ Does the recurrence cover every case without double-counting?

■ Are indices and loop boundaries consistent?

■ Does loop order accidentally reuse an item or overwrite a needed value?

■ Does the reported complexity include transition work?

■ Does a brute-force solution agree on small random inputs?

A practical testing tactic

For small n, write a simple exhaustive recursive solution that enumerates all choices. Compare it against the DP on many small inputs. This is differential testing: the brute-force version is too slow for large inputs but often easy to trust on tiny cases.

> **When a result is wrong, print the state table for a tiny input and manually verify what each entry means. A table of numbers is much more useful when every row and column has a clear interpretation.**

15. A Repeatable Workflow for Any DP Problem

Use this workflow in practice and interviews. Do not begin by memorizing a familiar code template.

1. Clarify the goal — Is it counting, feasibility, minimum, maximum, or reconstruction? Note constraints and edge cases.

2. Describe brute force — What choices could be made? Write a recursive solution conceptually.

3. Find repeated subproblems — Which calls ask the same question? What parameters uniquely identify that question?

4. Define the state — Write one sentence describing the exact meaning of dp parameters.

5. Derive the transition — Use a complete case split based on the first/last choice or a split point.

6. Set base cases — Handle empty input, minimum sizes, and impossible states.

7. Choose computation style — Use memoization or establish a valid tabulation order.

8. Analyze complexity — Count states, work per state, memory, and recursion depth.

9. Test tiny cases — Check boundaries and compare with brute force where possible.

10. Optimize only if needed — Compress memory or improve transitions after correctness is established.

Interview communication

Explain the state before the formula. Then explain why the transition covers all possibilities, why the order is valid, and how the complexity follows from the state count. This communicates reasoning rather than just pattern recognition.

> **If stuck, ask: “What must the final decision have been?” or “What information from the past changes the future?” Those questions often reveal the transition or the missing state dimension.**

16. DP Pattern Map and Final Cheat Sheet

Pattern map

Problem structure	Possible state shape	Natural order
A sequence / prefix	dp[i]	Increasing or decreasing i
Grid / path	dp[r][c]	Row/column topological order
Choice + capacity	dp[i][w]	Items and resource amount
Two strings / sequences	dp[i][j]	Increasing prefix lengths
Contiguous interval	dp[l][r]	Increasing interval length
Tree / hierarchy	dp[node][condition]	Postorder over children
Small subset	dp[mask] or dp[mask][last]	By subset size or mask dependencies

The five questions to remember

• What exactly does each state mean?

• What choices or previous states lead to it?

• What are the base cases?

• In what order can all dependencies be computed?

• How many states exist, and how much work does each take?

If you can answer these five questions clearly, you have the core structure of a DP solution. Practice applying the framework to different patterns; the goal is to learn the reasoning, not memorize isolated recurrences.

End of guide